

Procedimiento de analisis estatico profundo de muestras Excel maliciosas

Formatos .xls, .xlsx, .xlsm, .xlsb y .xlam

0. Flujo general

identificacion -> formato Excel -> estructura contenedora -> macros/objetos -> ofuscacion -> extraccion -> analisis

El objetivo de este procedimiento es analizar muestras Excel sospechosas de forma ordenada, sin ejecutar el documento y sin asumir que el primer objeto extraido contiene el payload principal. En Excel malicioso, la logica puede estar repartida entre macros VBA, macros Excel 4.0/XLM, objetos OLE, relaciones externas, objetos embebidos, hojas ocultas, cadenas en `sharedStrings.xml`, blobs raw o payloads descargados.

1. Informacion general de la muestra

1.1 Identificacion del fichero

En primer lugar se confirma el tipo real del fichero:

```
file muestra.xls
file muestra.xlsm
file muestra.xlsx
file muestra.xlsb
```

El objetivo es determinar si el documento es un Excel OLE clasico, un OOOXML/ZIP, un Excel binario o un fichero con extension falsa.

1.2 Hashes y footprinting

Se calculan los hashes principales:

```
md5sum muestra.xlsm
sha1sum muestra.xlsm
sha256sum muestra.xlsm
ls -lh muestra.xlsm
```

Estos valores permiten documentar la muestra, compararla con informes externos y asegurar que todos los analisis se realizan sobre el mismo artefacto.

1.3 Consultas externas

De forma opcional se pueden consultar VirusTotal, Joe Sandbox, Hybrid Analysis, MalwareBazaar u otros servicios. Estos resultados deben utilizarse como contexto o hipotesis de trabajo, no como sustituto del analisis manual.

2. Identificacion del formato Excel

Antes de elegir herramientas hay que diferenciar el tipo de contenedor:

Formato	Estructura	Herramientas iniciales
.xls / .xla	OLE/CFBF clasico	oleid, oledump.py, olevba, exiftool

Formato	Estructura	Herramientas iniciales
.xlsx	OOXML/ZIP sin macros VBA por diseño	unzip, grep, exiftool, oleid sobre embeddings
.xlsm	OOXML/ZIP con posible vbaProject.bin	unzip, olevba, oledump.py sobre vbaProject.bin
.xlsb	Excel Binary Workbook dentro de ZIP/OOXML	unzip, olevba, herramientas BIFF/XLSB
.xlam	Add-in de Excel con macros	olevba, unzip u oleid segun formato

La metodologia general es la misma que para otros documentos Office, pero las rutas internas y las herramientas cambian segun sea OLE clasico u OOXML/ZIP.

3. Analisis estatico inicial

3.1 Metadatos con ExifTool

```
exiftool muestra.xlsm > exiftool.txt
```

Se revisan autor, fechas, aplicacion generadora, tamanos, advertencias de parser, presencia de macros, rutas internas, objetos embebidos y campos anormales. Los metadatos no demuestran actividad maliciosa por si solos, pero orientan el analisis.

3.2 Analisis inicial de OLE clasico (.xls)

```
oleid muestra.xls
oledump.py muestra.xls
oledump.py --storages muestra.xls
olevba muestra.xls
```

En .xls se buscan streams, storages, macros VBA, hojas con macros XLM, objetos incrustados, entradas CompObj, Ole, Package y posibles datos binarios.

Si oledump.py muestra streams sospechosos, se extraen para analizarlos individualmente:

```
oledump.py -s NUMERO -d muestra.xls > stream_NUMERO.bin
oledump.py -s NUMERO -v muestra.xls > macro_NUMERO.vba
```

3.3 Analisis inicial de OOXML (.xlsx, .xlsm, .xlsb)

```
unzip -l muestra.xlsm
mkdir xlsx_extract
unzip -q muestra.xlsm -d xlsx_extract
find xlsx_extract -type f | sort
```

En OOXML se revisa la estructura interna del ZIP. Los elementos de mayor interes suelen estar en:

```
xl/vbaProject.bin
xl/worksheets/*.xml
xl/sharedStrings.xml
xl/externalLinks/*
xl/embeddings/*
xl/activeX/*
customXml/*
_rels/.rels
xl/_rels/workbook.xml.rels
```

3.4 Relaciones externas y objetos vinculados

```
grep -Rai 'TargetMode="External"' xlsx_extract/
grep -Rai "http|https|file:|\\\\\\\\" xlsx_extract/
```

Las relaciones externas son importantes porque un documento puede no contener el payload directamente, sino cargarlo desde una URL, una ruta de red o un recurso remoto. En OOXML estas referencias suelen aparecer en ficheros `.rels`.

4. Macros VBA y macros Excel 4.0/XLM

4.1 Analisis de macros VBA

```
olevba muestra.xlsm
olevba muestra.xls
olevba xlsm_extract/xl/vbaProject.bin
```

Se buscan eventos de autoejecucion, creacion de objetos COM, ejecucion de comandos, descarga de ficheros, escritura en disco y persistencia.

Palabras clave relevantes:

```
Auto_Open
Workbook_Open
Worksheet_Activate
CreateObject
WScript.Shell
Shell.Application
cmd.exe
powershell
mshta
regsvr32
rundll32
MSXML2.XMLHTTP
WinHttp.WinHttpRequest.5.1
ADODB.Stream
URLDownloadToFile
Environ
Application.Run
```

Si hay macros ofuscadas, se deben extraer y conservar tanto la version original como una version comentada o desofuscada. No se debe ejecutar la macro fuera de un entorno controlado.

4.2 Macros Excel 4.0 / XLM

Las macros XLM pueden estar en hojas ocultas o muy ocultas. Son especialmente relevantes en documentos Excel maliciosos porque permiten ejecutar funciones antiguas de Excel sin usar VBA convencional.

```
olevba muestra.xlsm
olevba muestra.xls
# Si se dispone de XLMMacroDeobfuscator:
# python XLMMacroDeobfuscator.py --file muestra.xlsm
```

Funciones a revisar:

```
AUTO_OPEN
EXEC
CALL
REGISTER
RUN
FORMULA
FORMULA.FILL
GET.WORKBOOK
GET.CELL
CHAR
CONCATENATE
MID
LEFT
```

RIGHT
HYPERLINK
WEBSERVICE

Si aparecen hojas ocultas, formulas largas, cadenas `CHAR()` concatenadas o llamadas `EXEC/CALL`, se debe priorizar la desofuscacion de XLM.

4.3 VBA stomping y p-code

En muestras avanzadas puede existir discrepancia entre el codigo fuente VBA visible y el p-code compilado. Esto se conoce como VBA stomping. Si el codigo visible parece inocuo pero la muestra se comporta de forma sospechosa, conviene analizar p-code.

Herramientas utiles si estan disponibles:

```
pcodedmp vbaProject.bin  
pcode2code vbaProject.bin
```

La ausencia de codigo sospechoso visible no descarta actividad maliciosa si hay indicadores de p-code anomalo o macros compiladas alteradas.

5. Deteccion de ofuscacion

La ofuscacion en Excel suele aparecer como cadenas concatenadas, arrays de numeros, funciones `CHAR()`, Base64, hexadecimal, XOR, datos en celdas ocultas, nombres definidos, hojas ocultas, relaciones externas o blobs embebidos.

5.1 Busqueda inicial de indicios

```
strings -a muestra.xlsm | head  
strings -a muestra.xlsm | grep -Ei "http|https|exe|dll|shell|cmd|powershell|wscript|mshta|CreateObject|Workb  
grep -Rai "http|https|cmd|powershell|wscript|mshta|exe|dll|CreateObject|Shell" xlsx_extract/
```

5.2 Cabeceras y datos embebidos

```
grep -Roba "MZ" xlsx_extract/  
grep -Roba "DOCF11E0" xlsx_extract/  
grep -Roba "PK" xlsx_extract/
```

Tambien se deben buscar cabeceras codificadas como texto hexadecimal:

```
grep -Rai "4d5a" xlsx_extract/          # MZ en hex textual  
grep -Rai "d0cf11e0" xlsx_extract/      # OLE/CFBF en hex textual
```

5.3 Comprobacion de ofuscacion avanzada

```
strings -a archivo  
strings -a -el archivo  
floss archivo  
binwalk archivo  
xxd archivo | less
```

Se buscan codificaciones como Base64, hexadecimal textual, UTF-16LE, URL encoding, secuencias `\xNN`, XOR simple, zlib o cadenas partidas.

```
grep -aEo "([A-Za-z0-9+/]{40,}={0,2})" archivo  
grep -aEo "(\\x[0-9a-fA-F]{2}){10,}" archivo  
grep -aEo "(%[0-9a-fA-F]{2}){10,}" archivo  
xorsearch archivo "http"  
xorsearch archivo "kernel32"
```

6. Extraccion de objetos y artefactos

6.1 Extraccion desde .xls OLE

```
oledump.py muestra.xls
oledump.py -s NUMERO -d muestra.xls > stream_NUMERO.bin
oledump.py -s NUMERO -v muestra.xls > macro_NUMERO.vba
```

Cada stream extraido debe analizarse de forma individual. No se debe asumir que el stream mas grande es el payload principal.

6.2 Extraccion desde .xlsm / .xlsx OOXML

```
mkdir xlsx_extract
unzip -q muestra.xlsx -d xlsx_extract
find xlsx_extract -type f | sort
```

Rutas habituales de interes:

```
xl/vbaProject.bin
xl/worksheets/*.xml
xl/sharedStrings.xml
xl/externalLinks/*
xl/embeddings/*
xl/activeX/*
customXml/*
_rels/.rels
xl/_rels/workbook.xml.rels
```

6.3 Validacion de artefactos extraidos

```
file xlsx_extract/xl/embeddings/*
md5sum xlsx_extract/xl/embeddings/*
xxd -l 64 xlsx_extract/xl/embeddings/*
```

Se buscan cabeceras OLE, PE, ZIP, scripts o blobs raw. Si una cabecera aparece desplazada, se puede carrear el contenido a partir del offset adecuado.

```
grep -aob "$(\printf '\xd0\xcf\x11\xe0\xa1\xb1\x1a\xe1')" objeto.bin
dd if=objeto.bin of=objeto_puro.ole bs=1 skip=OFFSET status=none
```

7. Analisis de OLE, OOXML y artefactos embebidos

7.1 Objetos OLE

```
oleid objeto.ole
oledump.py objeto.ole
oledump.py --storages objeto.ole
pecheck.py -l P objeto.ole
```

Si solo aparece Root Entry, el objeto puede estar vacio, corrupto, parcial o construido para cumplir estructura. Si aparecen streams con datos, deben extraerse y analizarse individualmente.

7.2 vbaProject.bin

```
file xlsx_extract/xl/vbaProject.bin
oleid xlsx_extract/xl/vbaProject.bin
oledump.py xlsx_extract/xl/vbaProject.bin
olevba xlsx_extract/xl/vbaProject.bin
```

vbaProject.bin es un contenedor OLE dentro del ZIP OOXML. Puede analizarse igual que un OLE tradicional.

7.3 Objetos embebidos

```
find xlsx_extract/xl/embeddings -type f -maxdepth 1 -print
file xlsx_extract/xl/embeddings/*
strings -a xlsx_extract/xl/embeddings/* | head
```

Los objetos embebidos pueden contener documentos, scripts, OLE, binarios, paquetes, accesos directos o payloads. Cada uno debe clasificarse y analizarse según su tipo real.

8. Búsqueda generica de cadenas en todos los artefactos

```
for f in $(find xlsx_extract -type f); do
    echo
    echo "===== $f ====="
    echo "[ASCII]"
    strings -a -t x -n 4 "$f" | grep -Ei "http|https|www\.|\.exe|\.dll|cmd|powershell|shell|download|url|kernel"
    echo "[UTF-16LE]"
    strings -a -el -t x -n 4 "$f" | grep -Ei "http|https|www\.|\.exe|\.dll|cmd|powershell|shell|download|url|kernel"
done
```

El objetivo es descubrir cadenas relevantes sin partir de IOCs conocidos previamente. Deben revisarse APIs, URLs, rutas, dominios, comandos, nombres de DLL, objetos COM y referencias a descarga o ejecución.

9. Correlacion con la estructura original

Una vez encontradas cadenas en artefactos extraídos, se debe localizar donde aparecen dentro de la estructura original del documento.

En .xls OLE, la correlación se realiza contra streams y storages identificados por oledump.py. En .xlsx / .xlsm, se realiza contra rutas internas del ZIP y ficheros .rels, worksheets, sharedStrings.xml, embeddings o vbaProject.bin.

```
grep -Rai "cadena_observada" xlsx_extract/
grep -Rai 'TargetMode="External"' xlsx_extract/
oledump.py muestra.xls | less
```

Lo relevante no es solo encontrar una cadena, sino determinar en qué estructura aparece, si está en una macro, en una celda, en una relación externa, en sharedStrings.xml, en un objeto embebido o dentro de un blob raw.

10. Analisis de contexto hexadecimal y shellcode

```
xxd -g 1 -s OFFSET -l 0x200 artefacto.bin
```

Si aparecen APIs de bajo nivel, rutas, URLs o nombres de DLL mezclados con bytes no imprimibles, se debe revisar el contexto. Se buscan patrones de shellcode:

```
call + datos inline
pop reg
jmp/call/pop
pushad / popad
mov eax, [fs:...]
```

acceso a PEB

```
mov eax, [edx+0x3c]
```

recorrido de export table

LoadLibrary / GetProcAddress

VirtualAlloc / VirtualProtect

Si hay acceso a estructuras PE, parsing de export table o cadenas inline, puede tratarse de shellcode position-independent. Si aparecen WinInet, WinHttp, UrlMon o URLDownloadToFile, puede tratarse de un downloader.

11. Analisis con desensamblador

Los blobs sospechosos pueden cargarse en Ghidra, IDA Free, Cutter o radare2 como binario raw.

En Ghidra:

```
Import as Raw Binary
Language: x86 32-bit little endian, si parece shellcode x86
Base address arbitraria, por ejemplo 0x0
Ir al offset sospechoso
Forzar disassembly
Crear funcion manualmente
Marcar cadenas inline
```

Se buscan rutinas de resolucion dinamica, parsing de PEB, parsing de PE/export table, saltos sobre datos, llamadas indirectas y cadenas inline. Ghidra no detectara automaticamente todo si el artefacto no tiene cabecera PE, entry point ni secciones.

12. Clasificacion de hallazgos

Nivel	Ejemplos	Interpretacion
Estructurales	OLE, ZIP/OOXML, vbaProject.bin, embeddings, externalLinks, hojas ocultas	Describen la estructura y donde mirar
Tecnicos	VBA, XLM, LoadLibrary, GetProcAddress, CreateObject, Shell, URLDownloadToFile	Indican posibles capacidades
IOCs	URLs, dominios, IPs, rutas, nombres de ficheros, hashes	Indicadores accionables para deteccion o correlacion

No todo hallazgo tecnico es un IOC. Por ejemplo, `kernel32` o `GetProcAddress` son cadenas tecnicas; una URL, un dominio o una ruta de payload son IOCs mas directos.

13. Conclusion estatica prudente

La conclusion debe separar lo observado de lo inferido. No se debe afirmar que el documento descarga o ejecuta algo si solo se han encontrado cadenas. Debe indicarse que los elementos son compatibles con una determinada funcionalidad y que la ejecucion real debe confirmarse con emulacion o analisis dinamico.

Ejemplo:

El analisis estatico identifica un documento Excel con macros y objetos embebidos. En los artefactos extraidos

14. Cuando pasar a emulacion o analisis dinamico

Se pasa a emulacion o dinamico cuando el analisis estatico detecta shellcode, macros ejecutables, blobs raw, comandos, URLs, objetos COM de descarga o comportamiento potencialmente activo.

Herramientas utiles:

```
Speakeasy
scdbg
Unicorn
Qiling
x32dbg con loader, solo si merece la pena
Procmon
Process Explorer
Regshot
Wireshark
```

Para blobs raw:

```
speakeasy.exe -t blob.raw --raw -a x86 -o salida.json
```

```
speakeasy.exe -t blob.raw --raw -a x86 --raw_offset 0x100 -o salida_100.json
```

Para macros, el análisis dinámico debe realizarse en una VM aislada, con snapshots y monitorización de procesos, ficheros, registro y red. No se deben habilitar macros fuera del laboratorio.

Checklist final

- Confirmar tipo real de fichero: OLE, OOXML, XLSB, add-in o extensión falsa.
- Calcular hashes y guardar metadatos.
- Analizar macros VBA y XLM, si existen.
- Revisar relaciones externas, embeddings, ActiveX, **sharedStrings.xml** y hojas ocultas.
- Extraer y validar todos los artefactos relevantes.
- Buscar cadenas ASCII y UTF-16LE sin partir de IOCs conocidos.
- Correlacionar cadenas con la estructura original.
- Revisar contexto hexadecimal y patrones de shellcode.
- Clasificar hallazgos estructurales, técnicos e IOCs.
- Redactar conclusiones prudentes y pasar a emulación/dinámico si procede.